

3

Adversarial Search
and Games

3.1 : Game Theory, Optimal Decisions in Games, Heuristic Alpha-Beta Tree Search, Monte Carlo Tree Search, Stochastic Games, Partially Observable games, Limitations of Game Search Algorithms

Q.1 Answer the following question :
Minimax is not good for game playing when the opponent is not playing optimally. Justify using suitable example.

Ans. : • Minimax assumes a perfectly rational opponent, who also takes optimal actions. However, in practice, most human opponents depart from rationality. In this case, the best move at any given step may not be one that is indicated by Minimax and an algorithm that takes into consideration human imperfections will perform better.

- If the tree is too large to fully explore, then there is a possibility that a suboptimal move could take Max into an area of the tree he hadn't considered that is worse than he thought could happen, but as Stuart said in class, how to deal with suboptimal play in this setting is still an open research question. Most algorithms, for example chess programs, assume an optimal opponent, and it doesn't seem to hurt them very much—but that's an empirical, not theoretical, result.
- Optimality is still well defined, even if your opponent isn't playing well. Moreover, if the game tree is small enough that your agent can fully explore it, then the optimal player really doesn't care what the other one does.

- Let's say Max goes first. What will Max do? He will look at every possible game sequence. He will then take the action which guarantees that he will get a score of at least X. No matter what Min does in subsequent moves, Min can never get a score less than X. So Max's score will only go up if Min doesn't play well.

Q.2 We have two players: MIN who plays first and can make 4 moves. MAX who plays second and can make 2 moves. Suppose that after 1 turn, the values of the leaves are as in the Fig. Q.2.1 :

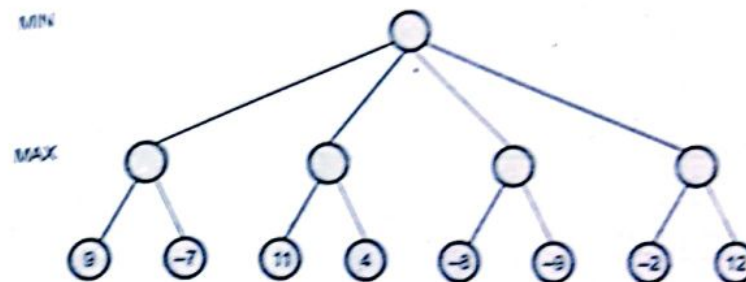


Fig. Q.2.1

Compute (with the algorithm minimax) the value of the root of the tree, then say which is the most convenient move for MIN. Then tell with the reason, which parts of the tree are not generated if we perform an alpha-beta pruning.

Ans. : Now, applying alpha-beta pruning algorithm,

Role of Beta (β) - For root A, the minimizer, value - 8, is assured by moving to node D. This value is compared with nodes B, C and E. This value of β is now used as reference point. Any node whose value is less than or equal to β value is acceptable else are rejected that is pruned. (As shown in the Fig. Q.2.1 (a))

Role of alpha (α) - for, root as D, the maximizer is assured by node having value - 8. This is compared with other leaf node. Any node

whose value is greater than or equal to α value is acceptable else those nodes are rejected. (As shown in the Fig. Q.2.1(a)).

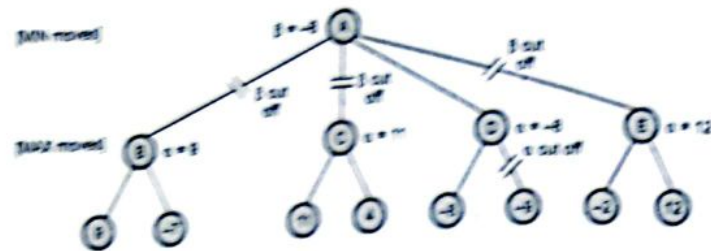


Fig. Q.2.1(a)

Q.3 Consider the game tree of Fig. Q.3.1 in which the static scores are from first player's point of view. Suppose the first player is maximizing player. Applying mini-max search, show the backed-up values in the tree. What move will the MAX choose? If the nodes are expanded from left to right, what nodes would not be visited using alpha-beta pruning.

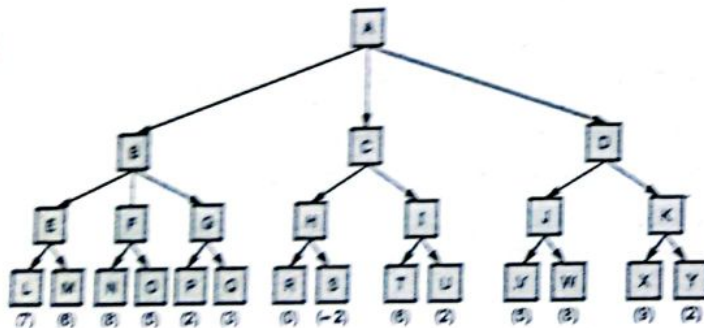
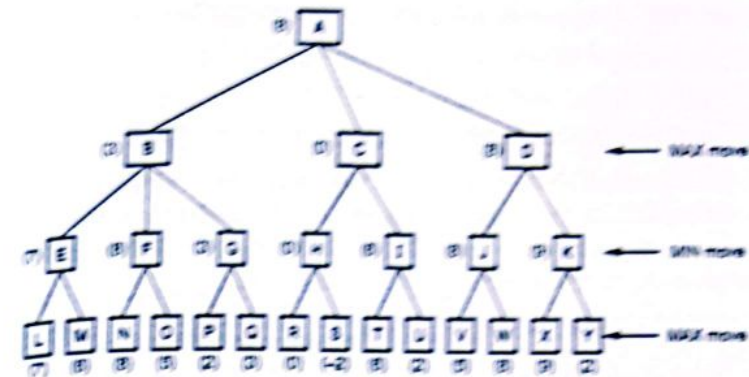
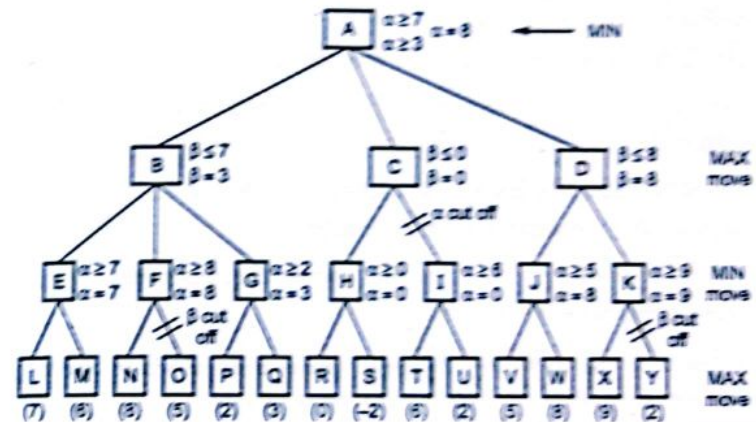


Fig. Q.3.1 Game tree

Ans. : Applying mini-max algorithm.



Applying Alpha-Beta pruning



Q.4 Explain Min-Max search procedure with an example.

Ans. :

1. Given a game tree the optimal strategy can be determined by examining the minimax value of each node.

2. The minimax value of a node is the utility (for player called MAX) of being in the corresponding state, assuming that both players play optimally from this stage to the end of the game.
3. The minimax value of a terminal state is just its utility.
4. Given a choice, MAX will prefer to move to a state of maximum value, whereas MIN prefers a state of minimum value.

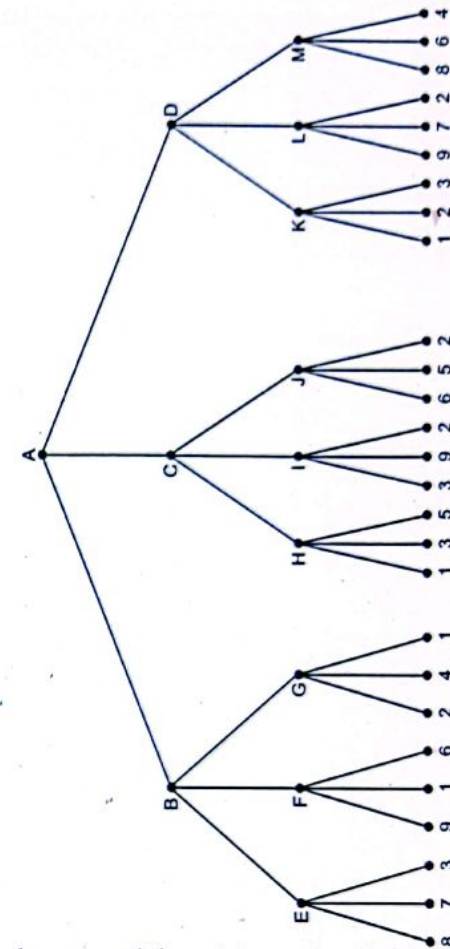
Mini-Max Algorithm : The minimax algorithm computes the minimax decision from the current state. It is used as a searching technique in game problems. The minimax algorithm performs a complete depth-first exploration of the game-tree.

The Algorithm :

1. The start node is MAX (player 1) node with current board configuration.
2. Expand nodes down (play) to some depth of look-ahead in the game.
3. Apply evaluation function at each of the leaf nodes.
4. "Back up" values for each non-leaf nodes until computed for the root node.
5. At MIN (player 2) nodes, the backed up value is the minimum of the values associated with its children.
6. At MAX nodes, the backed up value is the maximum of the values associated with its children.

Q.5 Consider the following 2 player game tree in which static scores are given from the first player's point of view :

Suppose the first player is the maximizing player. What move should be chosen ? Why ? Use mini-max search to solve. Also explain limitations of mini-max search. How to overcome them question.

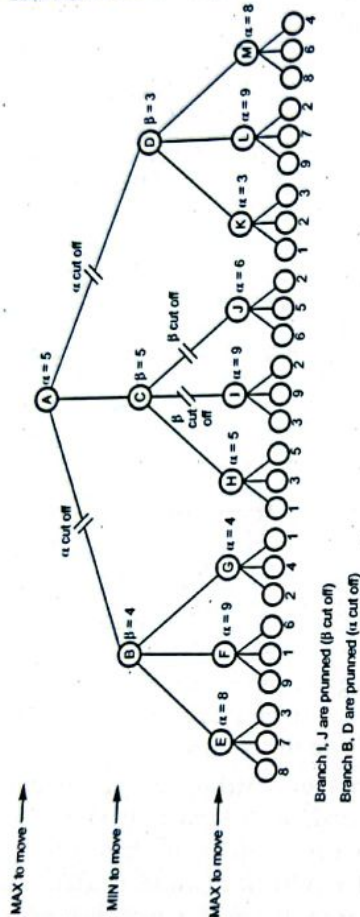


Ans. : A disadvantage of the minimax algorithm is that each board state has to be visited twice : One time to find its children and a second time to evaluate the heuristic value.

- During the process of searching for the next move, not every move (i.e. every node in the search tree) needs to be considered in order to reach a correct decision. In other words, if the move being considered results in a worse outcome than current best possible choice, then the first move that the opposition could

make which is less than the best move will be the last move that one need to look at, as the opposition will at least choose that move.

- Minimax tends to be too slow for games such as chess. For each turn, the player has many choices to decide on, the branching factor of a game of chess is huge and therefore the deeper we go, the slower it gets. On average, the branching factor for chess tends to 30. This is, 30 subtrees per turn are created.



- Alpha beta pruning method is better minimax version which overcomes the limitations of mini-max algorithm. Alpha beta reduces the number of nodes to be expanded and therefore we reduce the branching factor.

Q.6 Explain alpha-beta cut off search with an example. State a case when to do alpha pruning.

Ans. : α - β Pruning

- The problem with minimax algorithm search is that the number of game states it has to examine is exponential in the number of moves.
- α - β proposes to compute the correct minimax algorithm decision without looking at every node in the game tree.

Steps In Alpha-Beta Pruning

- MAX player cuts off search when he knows MIN-player can force a provably bad outcome.
- MIN player cuts off search when he knows MAX-player can force provably good (for MAX) outcome.
- Applying an alpha-cutoff means we stop search of a particular branch because we see that we already have a better opportunity elsewhere.
- Applying beta-cutoff means we stop search of a particular branch because we see that the opponent already has a better opportunity elsewhere.
- Applying both forms is alpha-beta pruning.

α - β Pruning Example :

Step 1 :

MAX move

MIN move

MAX move

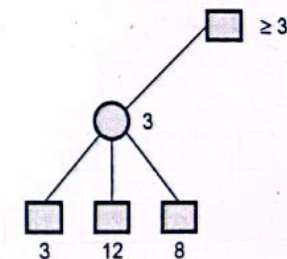


Fig. Q.6.1 Pruning example (i)

Step 2 :

MAX move

MIN move

MAX move

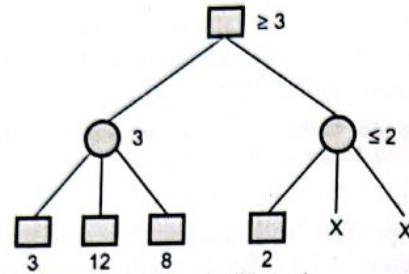


Fig. Q.6.2 Pruning example (ii)

Step 3 :

MAX move

MIN move

MAX move

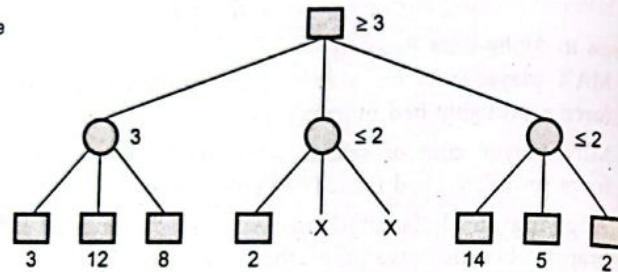


Fig. Q.6.3 Pruning example (iii)

Q.7 Explain the minimax procedure for game playing. Find out the values of the nodes starting from node A to O as shown in Fig. Q.7.1 using minimax procedure assuming that root node is max node.

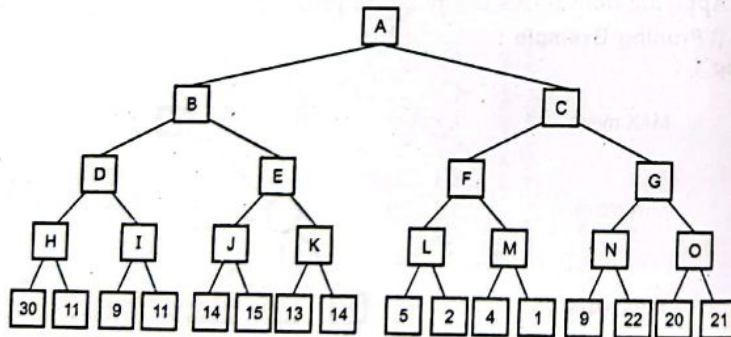


Fig. Q.7.1

Ans. : Mini-max procedure is a game playing strategy in which players adopt those strategies which will maximize their gains, while minimizing their losses.

- The minimax search procedure is a depth-first, depth-limited search procedure. The idea is to start at the current position and use the plausible - more generator to generate the set of possible successor positions. Now by applying the static evaluation function to those positions we can choose the best one. After doing so we can back that value upto the starting position to represent our evaluation of it. The starting position is exactly as good as the position generated by the best move we can make next.

- Assuming that the static evaluation function returns large value to indicate good situation for us, so our goal is to maximise the value of static evaluation function of the next board position. It is a simple look ahead strategy for two person game-playing.

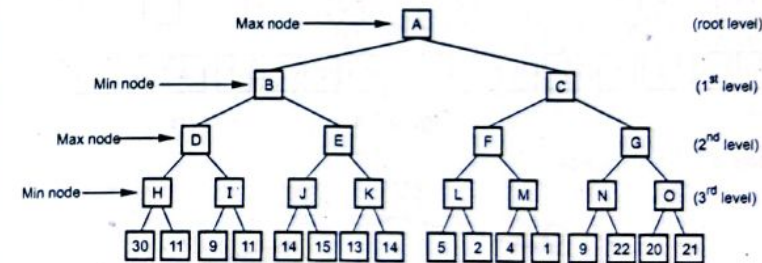


Fig. Q.7.2

First to move is max node.

∴ At 3rd level min will choose the value and minimum value would be chosen.

Step 1 : Min moves

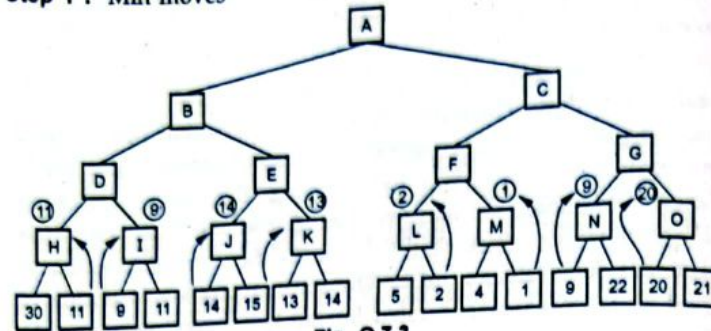


Fig. Q.7.3

Step 2 : Max moves

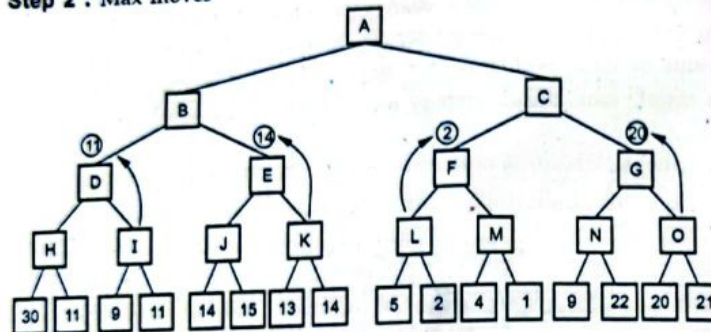


Fig. Q.7.4

Step 3 : Min moves

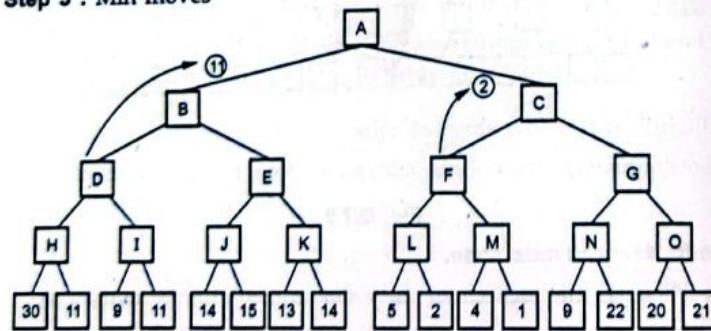


Fig. Q.7.5

Step 4 : Max moves

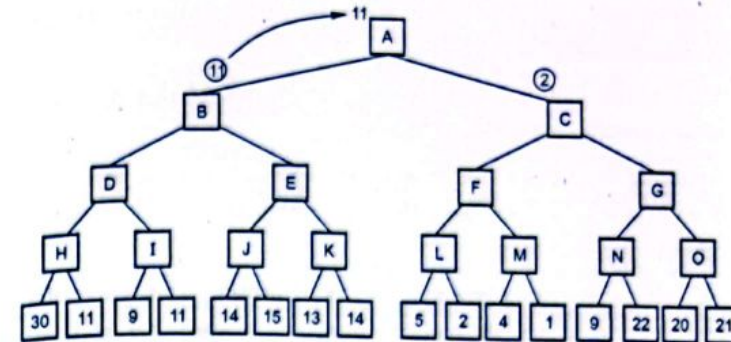


Fig. Q.7.6

Final value at root node is value 11.

Q.8 Explain α - β pruning procedure. Mark the nodes in the Fig. Q.8.1 which will prune out ?

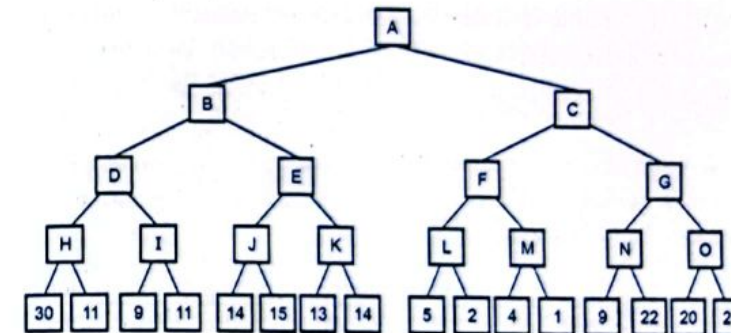


Fig. Q.8.1

Ans. : Alpha-Beta Pruning

1. The problem with minimax algorithm search is that the number of game states it has to examine is exponential in the number of moves.
2. α - β proposes to compute the correct minimax algorithm decision, without looking at every node in the game tree.

Steps in Alpha-Beta Pruning

1. MAX player cuts off search when he knows MIN-player can force a provably bad outcome.
2. MIN player cuts off search when he knows MAX-player can force provably good (for MAX) outcome.
3. Applying an alpha-cutoff means we stop search of a particular branch because we see that we already have a better opportunity elsewhere.
4. Applying beta-cutoff means we stop search of a particular branch because we see that the opponent already has a better opportunity elsewhere.
5. Applying both forms is alpha-beta pruning.

Solution to pruning for Fig. Q.8.1.

Role of Alpha (α) - For node A, the maximizer, a value of 11 is assured by moving to node B. We can compare this value with that of node C. A being maximizer would follow path of whose value is greater than 11. Hence, this value 8 being the least that a maximizing node can obtain, is set as the value of α .

- This value of α is now used as referenced point. Any node whose value is greater than α is acceptable and all nodes whose values are less than α are rejected.
- From Fig. Q.8.1 (After min max moves), it is known that C node has value 2 which is less than α value. Hence entire subtree under node C is completely rejected.

Role of Beta (β) - Now consider the tree headed by node B which is minimizer and the paths for expansion are chosen from the values at each leaf nodes. B being minimizer will choose D node which minimizes the value. The value of 11 is used as reference and is called as Beta (β). Any node whose value is less than that is acceptable and values more than that are rejected. The value of

Beta (β) is passed to E for comparing. The minimizer would be benefited by only moving to D than E. Hence entire tree under E is pruned.

Q.9 How can minimax also be extended for game of chance ?

Ans. : In a game of chance we can add extra level of chance nodes in game search tree. These nodes have successors which are the outcomes of random element.

The minimax algorithm uses probability P attached with chance node d_i based on this value. Successor function $S(N, d_i)$ give moves from position N for outcome d_i .

Q.10 What is game ?

Ans. : • The term game means a sort of conflict in which n individuals or groups (known as players) participate.

- Game theory denotes games of strategy.
- John Von Neumann is acknowledged as father of game theory. Neumann defined game theory in 1928 and established the mathematical framework for all subsequent theoretical developments.
- Game theory allows decision-makers (players) to cope with other decision-maker (players) who have different purposes in mind. In other words, players determine their own strategies in terms of the strategies and goal of their opponent.
- Games are integral attribute of human beings. Games engage the intellectual faculties of humans.
- If computers are to mimic people they should be able to play games.

Q.11 What is game tree ?

Ans. : The game tree : The initial state and the legal moves for each side, define the game tree for the game.

• Description of the game tree.

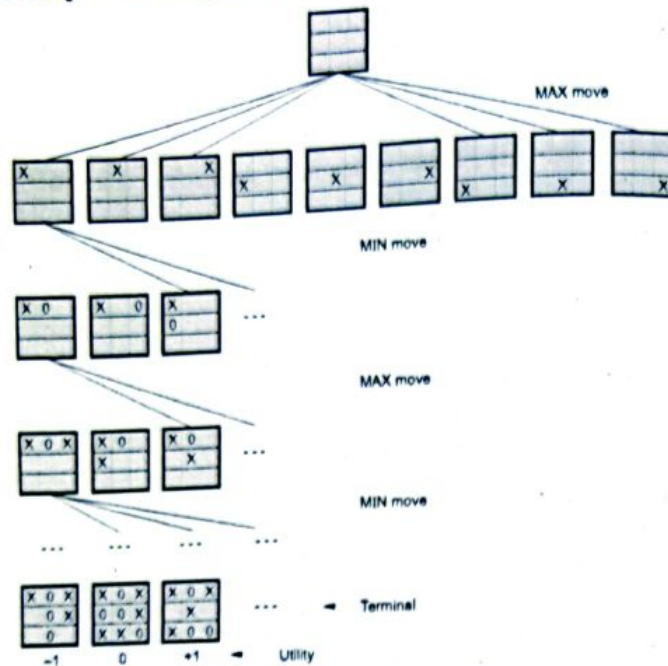


Fig. Q.11.1 Example of game tree

1. Root node -

Represents board configuration and decision, required as to what is the best single next move.

If my turn to move, then the root is labeled a MAX node indicating it is my turn;

Otherwise it is labeled a MIN, node to indicate it is my opponent's turn.

2. Arcs - Represent the possible legal moves for the player that the arcs emanate from.

3. At each level, the tree has nodes that are all MAX or all MIN.

4. Since moves alternate, the nodes at level 'i' are of opposite kind from those at level $i + 1$.

Q.12 What are properties of mini-max ?

Ans. : Properties of Mini-Max

1. Minimax provides a complete solution for finite tree.
2. Minimax provides optimal strategy against an optimal opponent.
3. The time complexity is $O(b^m)$.
4. The space complexity is $O(bm)$ for depth-first exploration where algorithm generates all successor at once, or $O(m)$ for an algorithm that generate successors one at a time.

Q.13 What is a made-up game ? What is ply ?

Ans. : The 'made-up' Games and Concept of 'ply'.

- For study purpose we can form a game a tree which is constructed up to certain level (i.e. upto certain depth). This is called as a made-up game. The term ply refers to depth of the tree.

- For example - ply 4 is level at depth 4 below the root node.

Example

A two ply game tree, for game of tic-tac-toe.

1. Assume that two players named MIN and MAX who are playing the game.
2. MAX is playing first.
3. The possible moves for MAX at the root node are labeled a_1 , a_2 and a_3 .
4. The possible replies to a_1 for MIN are b_1 , b_2 , b_3 and so on.
5. This particular game ends after one move each by MAX and MIN.
6. In game parlance, we say that this tree is one which, moves deep consisting of two-half-moves, each of which is called a ply.

3.2 : Constraint Satisfaction Problems, Constraint Propagation - Inference in CSPs, Backtracking, Search for CSPs

Q.14 Solve the following crypt arithmetic problem.

DONALD
+ GERALD
ROBERT

Ans. :

	6	5	4	3	2	1
	D	O	N	A	L	D
+	G	E	R	A	L	D
c1	c2	c3	c4	c5		
	R	O	B	E	R	T

- Let initially 'D=5', so 'D+D=T' therefore 'T=0' & 'c5=1'.
- In column 5 'O+E=O' as 'T=0' so E cannot be 0, therefore 'E=9'. 'O+9=O' is possible if 'c2=1'. Therefore 'c2=1'.
- In column 3 'A+A=9', but addition of any 2 same number is always even, given that addition is 9 which is possible when there is carry. Therefore 'c4=1', so 'A=4'.
- Remaining numbers to be assigned are {1,2,3,6,7,8} to {O,N,R,B,L,G}.
- We have 'E=9' & 'c2=1' so from column 5 we get 'c1=1'. Also from column 2 we have 'L+L+c5=R' where 'c5=1' therefore R is odd so R can be 1 or 3 or 7. As 'D+G' does not generate carry shown in column 6 so R cannot be 1 or 3. Therefore 'R=7' & 'G=1'.
- We have 'R=7' so from column 2 we have 'L+L+1=17', therefore 'L=8'.
- From column 3 we get that 'A+A+c4=E' and so there is no carry, therefore 'c3=0'.
- From column 4 we get 'N+R+c3=B' we have R=7 & 'c3=0', so 'N+7=10+B', therefore 'N=B+3'. {2,3,6} are remaining to be assigned so to satisfy the constraint 'N=B+3' we get 'B=3' & 'N=6'.

9. And remaining 'O=2'.

SOLUTION :

	5	2	8	4	8	5
+	1	9	7	4	8	5
	7	2	3	9	7	0

VALUES :

D = 5
O = 2
N = 6
A = 4
L = 8
G = 1
E = 9
R = 7
B = 3
T = 0

Q.15 Solve following crypt-arithmetic problem with appropriate strategy/steps :

		E	A	T
+	T	H	A	T
A	P	P	L	E

Ans. :

EAT
+THAT
APPLE

Here we know that T is not 0, as T+T is not equal to T. The case is same with A. After that we have to solve every letter for the problem making each letter unique. The steps are below.

EA9

+ 9HA9

APPLE

8A9

+ 9HA9

APPL8

819

+ 9H19

APPL8

819

+ 9H19

APP38

819

+ 9219

A0038

819

+ 9219

10038

FINAL ANSWER : EAT = 819, THAT = 9219, APPLE = 10038

Values :

E = 8

A = 1

T = 9

H = 2

P = 0

L = 3

Q.16 Discuss AO* algorithm. Give one example where AO* is suitable to apply.

Ans. : • AO* is useful for searching game trees, problem solving etc. but in most cases more domain specific search algorithms (e.g. alpha-beta pruning for game trees, general or domain specific planning algorithms) are used instead.

- In particular, AI uses knowledge-intensive approaches and in practical applications heavy use is made of domain specific knowledge or problem conditions to produce better (faster or more optimal solutions).

- Game search is an example where full-breadth search is standard, but this may be because of the small (relative to other domains) size of the search space. Even in game tree search, extensive use is made of problem specific features, i.e. often search is terminated only in quiescent states (i.e. not during a forced exchange or when there is a check).

- In planning, often knowledge is used to guide a search of a generated solution space rather than doing a state space search. This gives non-optimal solutions but for many domains it yields reasonable solutions at much less cost.

Q.17 Solve the following Crypt arithmetic problem :

BASE
+ BALL

GAMES

Ans. :

BASE

+ BALL

GAMES

There are seven distinct digits from 10 preliminary digits that are from [0-9]: A, B, S, E, L, G, M.

As we are just adding 2 numbers so possible carry overs are either 1 or 0.

But, when we are adding B+B it gives us some carry that is greater than zero as both 4 digit numbers add up to form a five digit number.

Hence, G can't be zero.

So, $G = 1$

Now our question will look somewhat like this :

```

BASE
+ BALL
-----
1 AMES

```

Then,

if 2 B's are added and they are giving us value greater than equal to 10

Then, B must be greater than equal to 5

Consider $B = 9$ first

if $B = 9$ then $A = 8$ which is not possible as then $B + B + 1$ is not equal to A

if $B = 8$ then $A = 6$ which is not possible for same reason

Take $B = 7$ then $A = 4$ which is possible as it is not leaving any carry over.

So,

$M = A + A + x$ where x is carry over from $S + L + y$ where y is carry over from $E + L$

knowing the fact that x and y can take maximum value of 1 and minimum value of 0

We can say

M should be either 8 or 9

8 when $x = 0$

9 when $x = 1$

Assuming $M = 8$

that is $x = 0$

$S + L + y = E$

$E + L = 10 * y + S$

$S + L + y + L = 10 * y + S$

$2 * L = 9 * y$

where y can take either 0 or 1

if $y = 0$ then $L = 0$ then S and E are not distinct which is not possible according to question.

if $y = 1$

then $L = 4.5$ which is again not possible as L must be a single digit among 0 to 9

So, $M = 8$ is not possible

So, our question further reduces to 1

```

74SE

```

```

+ 74LL
-----

```

```

1 49ES

```

Now,

$S + L = 10 * 1 + E = 10 + E$, and

$E + L = 10 * y + S$

So,

here y can take maximum and minimum values of 1 and 0 respectively.

So,

if $y = 0$

Then,

$$E + L = S, \text{ or } S - L = E, \quad \dots (1)$$

and

$$S + L = 10 + E \quad \dots (2)$$

Adding both we get,

$$2*S = 2*E + 10$$

or

$$S = E + 5$$

Subtracting 1st equation from 2nd we get

$$2*L = 10$$

$$\text{or } L = 5$$

So, our question further reduces to 1 0

$$74SE$$

$$+ 7455$$

$$149ES$$

So,

$$E + 5 = S, \text{ and}$$

$$S + 5 + 0 = E + 10*1$$

$$S + 5 = E + 10$$

$$S = E + 5$$

Now we have to choose digits from 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9 other than 1, 4, 7, 9 and 5.

So, we can take values of E and S from 0, 2, 3, 6, 8 such that $S = E + 5$

0 is there 5 can't be chosen

2 is there 7 can't be chosen

6 is there 11 can't be chosen

8 is there 13 can't be chosen

So, only possible pair left is 3 and 8 which satisfy our constraints.

Hence,

$$S = 8 \text{ and } E = 3$$

$$10$$

$$7483$$

$$+ 7455$$

$$14938$$

So,

$$A = 4,$$

$$B = 7,$$

$$S = 8,$$

$$E = 3,$$

$$L = 5,$$

$$G = 1, \text{ and}$$

$$M = 9$$

Q.18 Solve the crypt - arithmetic problem with the following constraints. Give solution steps.

Constraints :- i) Use decimal arithmetic ii) No two letters possess same digit

$$\begin{array}{r} C R O S S \\ + R O A D S \\ \hline D A N G E R \end{array}$$

OR Solve the following in detail

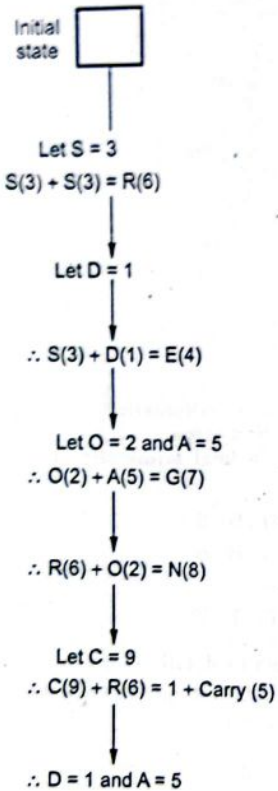
$$\begin{array}{r} C R O S S \\ + R O A D S \\ \hline D E N G E R \end{array}$$

Ans. : Hence the solution,

9	6	2	3	3
6	2	5	1	3
2	5	8	7	4
6				

where

C = 9,	R = 6,	O = 2,
S = 3,	D = 1,	A = 5,
G = 7,	N = 8,	E = 4



Q.19 What do you mean by constraint satisfaction problems ? Explain constraint propagation algorithm using suitable example.

- Ans. : 1. A constraint satisfaction problem is defined by a set of variables $X_1, X_2, \dots, X_n$ and a set of constraints $C_1, C_2, \dots, C_m$.
2. Each variable X_i has a nonempty domain D_i of possible values.
3. Each constraint C_i involves some subset of the variables and specifies the allowable combinations of values for that subset.
4. A state of the problem is defined by an assignment of values to some or all of the variables. $\{X_i = V_i, X_j = V_j, \dots\}$.
5. An assignment that does not violate any constraints is called a consistent or legal assignment.
6. A complete assignment is one in which every variable is maintained and a solution to a CSP is, a complete assignment that satisfies all the constraints.
7. Some CSPs also require a solution that maximizes an objective function.

Consider graph colouring problem as shown in Fig. Q.19.1. Constraints are -

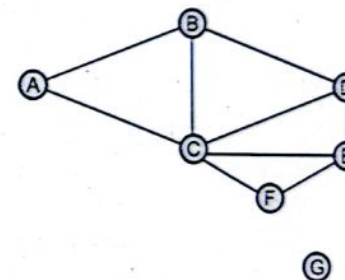


Fig. Q.19.1 Graph colouring problem

1. We have three colours for colouring a vertex.
2. No two adjacent vertices have same colour.

Given three colours-(Red, Green, Blue). Allowable combination for A, B vertices would be,

$\{(R, G), (R, B), (G, R), (G, B), (B, R), (B, G)\}$

Constraint Propagation

1. A combined approach of heuristic plus forward checking gives more reliable, accurate and efficient results than a singular approach.
2. The forward checking propagates information from assigned to unassigned variables but can not avoid or detect all failure.
3. Constraint propagation repeatedly enforces constraints locally.
4. The idea of arc consistency provides a fast method of constraint propagation that is substantially stronger than forward checking. Here "arc" refers to a directed arc in the constraint graph.

Constraint Propagation using Arc Consistency

1. It is fast method of constraint propagation.
2. $X \rightarrow Y$ is consistent if (for every value of X there is some allowed value Y . For example $[V_2 \rightarrow V_1]$, is consistent iff $V_1 = \text{Red}, V_2 = \text{Blue}$] (i.e. for every value of x in X there is some allowed value y in Y). This is directed property example - $[V_1 = V_2]$

$V_1 = V_2$ is consistent iff

$V_1 = \text{Red and } V_2 = \text{Blue}$

3. As directed arcs between variables represent the domains of specified variables, they are consistent with each other.
4. Constraint propagation can be applied as preprocessing or propagation step.
 - i) Before search-preprocessing.
 - ii) After search-propagation.
5. The procedure for maintaining arc consistency, can be applied repeatedly.

Constraint propagation using K-consistency

1. Arc consistency is not capable of detecting all inconsistencies. Partial assignments $[V_1 = \text{Red}, V_2 = \text{Red}]$ are inconsistent.
2. K-consistency is very strong form of constraint propagation.
3. A CSP is K-consistency if for any set of $K-1$ variables and for any consistent assignment to those variable a consistent value can always be assigned to any K^{th} variable.

END... ✍